



Riverpod State Management

From Provider to Riverpod: Understanding the evolution and why Riverpod solves critical state management problems

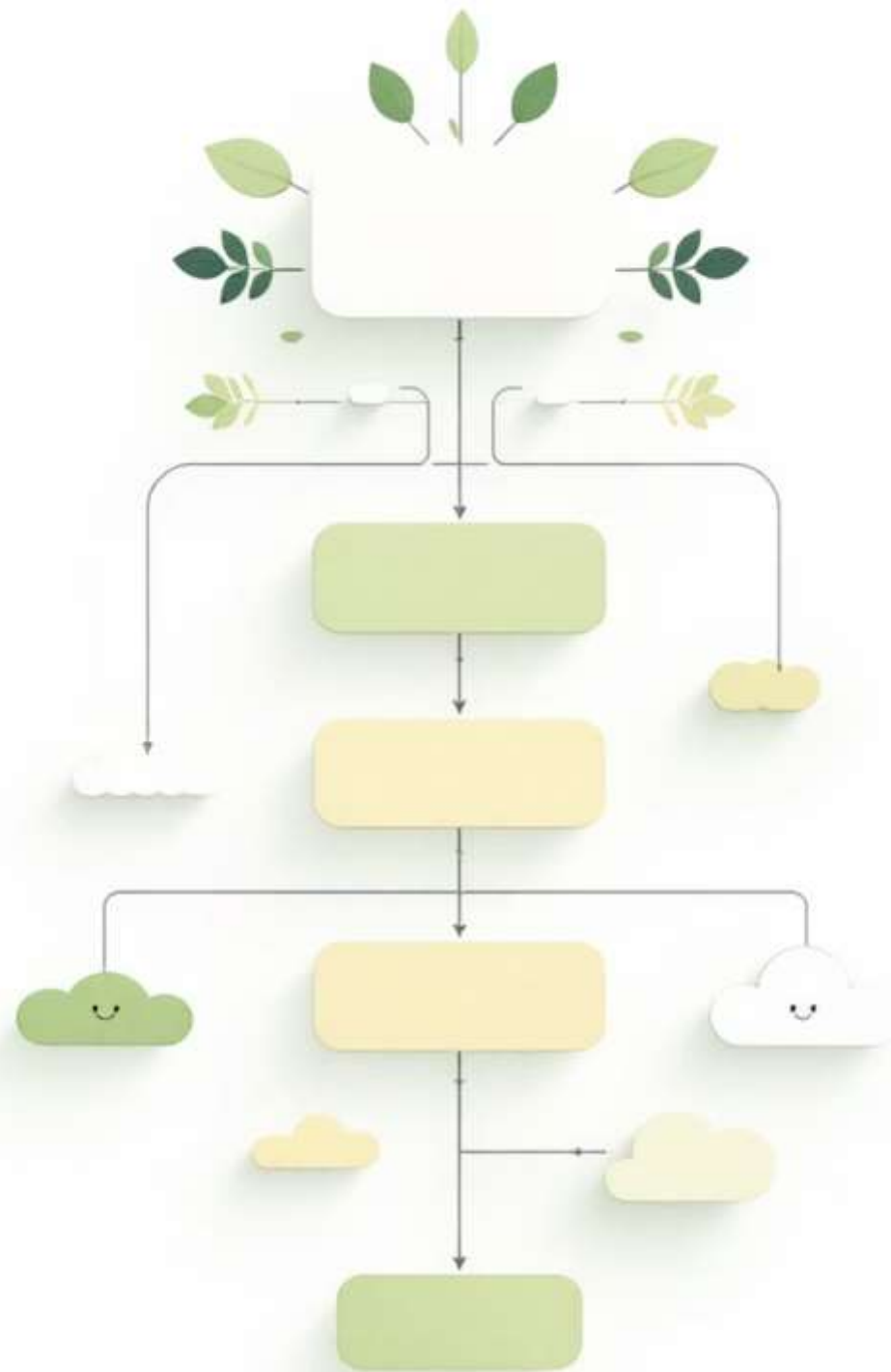
What is Riverpod?



The Same Author, Better Solution

Riverpod was created by Remi Rousselet, the same developer behind Provider. The name is literally an anagram of "PROVIDER".

Riverpod wasn't created from scratch—it evolved to fix the limitations that developers encountered with Provider in real-world applications.



Problem 1: Provider Depends on BuildContext

The Issue

With Provider, you always needed a `BuildContext` to read state. This meant you couldn't access state outside of widgets or in pure Dart classes.

Why It Matters

Business logic classes should not need `BuildContext`—that's a UI concern, not a logic concern. Forcing context into service classes breaks separation of concerns.

Problem 2: Runtime Errors Only

Compile-Time vs Runtime

Provider could only throw errors at runtime, not during compilation. If you forgot to declare a provider above in the widget tree, you wouldn't know until the app crashed.

The Impact

You'd discover bugs only when users navigated to specific screens—not during development. This made debugging painful and led to production crashes that could have been caught earlier.



Problem 3: No Multiple Providers of Same Type

The Limitation

Provider couldn't distinguish between two providers of the same type. Only one could exist at a time, and the second one would silently override the first.

Real-World Need

Real apps often need multiple instances of the same service—like two different API clients, two database connections, or separate configurations for development and production.

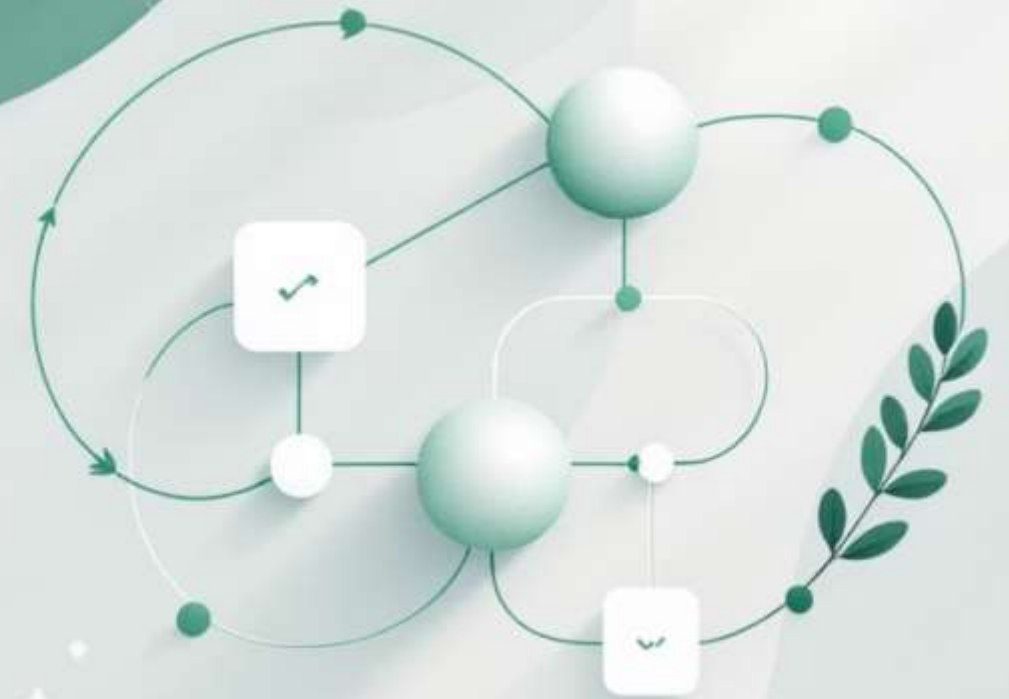
Problem 4: Providers Couldn't Depend on Each Other Easily

ProxyProvider Was the "Fix"

Combining or making one provider depend on another required `ProxyProvider`, which had a confusing API, edge cases, and felt like a hack.

Why It Was Fragile

The `ProxyProvider` API was awkward, required initial values, and rebuilt in unpredictable ways. It made dependency management unnecessarily complex.



Problem 5: No Auto Disposal / Lifecycle Control

Immediate Creation

Providers were created immediately on app start, even if the user never visited the screen that needed them. This wasted memory and resources.

Unpredictable Disposal

When widgets were removed from the tree, disposal was unpredictable, leading to memory leaks in complex navigation flows and poor performance.

Problem 6: Hard to Test

Testing Pain

Because Provider was tied to `BuildContext` and the widget tree, unit testing business logic required full widget setup just to test simple functions.

Excessive Setup

Just to test a simple cart function, you needed a complete widget tree with `MaterialApp`, providers, and entire UI structure. Way too much setup for a simple unit test.



Riverpod Solves All of This



No BuildContext

Uses `ref` instead of context—works anywhere in Dart code



Multiple Instances

Each provider is a unique variable—no conflicts between same types



Auto Dispose

Providers auto-dispose when no longer watched—no memory leaks



Compile-Time Safety

Global variables catch missing providers at compile time, not runtime



Clean Dependencies

`ref.watch()` inside providers makes dependencies effortless



Easy Testing

`ProviderContainer` lets you test without any widget tree



Getting Started with Riverpod

Setup

```
dependencies:  
  flutter_riverpod: ^2.5.1
```

ProviderScope

```
void main() {  
  runApp(  
    ProviderScope(  
      child: MyApp(),  
    ),  
  );  
}
```

Define Providers

```
class CounterNotifier extends Notifier<int>{  
  @override  
  int build(){  
    return 0;  
  }  
  void increment(){  
    state++;  
  }  
  void decrement(){  
    state--;  
  }  
  
  void reset(){  
    state=0;  
  }  
}  
  
final counterProvider =  
  NotifierProvider<CounterNotifier,int>((){  
    return CounterNotifier();  
  });
```



📌 **Key Takeaway:** Riverpod takes everything that made Provider popular and eliminates its pain points. The result is state management that's both powerful and a joy to work with.